

TransitOps

Consolidated Driver & Vehicle Rating Scores, Supporting Schema, and Other Algorithm Opportunities

This document defines a single 1–3 star rating for Drivers and a single 1–3 star rating for Vehicles, each built from a weighted blend of the metrics already defined in the KPI reference. It includes the database schema needed to compute and store them, and closes with a survey of other places in TransitOps where an algorithm (beyond assignment and rating) adds real value.

1. Driver Rating Score (1–3 stars)

A single star rating (displayed as ★ icons, half-star precision, scale of 1–3) the Safety Officer and Fleet Manager can both read at a glance, and that feeds directly into the assignment algorithm's "driver safety score" factor. Built from the same five normalized components (each scaled 0–1), then weighted, summed, and mapped onto a 3-point scale.

Component	Formula (normalized 0–1)	Weight	Why
Safety score	<code>Driver.safetyScore / 100</code>	0.30	Existing manually/telematics-set score is still the strongest single signal
License compliance	<code>1 if licenseExpiryDate > today+30, 0.5 if within 30 days, 0 if expired/suspended</code>	0.20	Hard compliance risk, not just a soft metric
Completion reliability	<code>COUNT(Trip WHERE status=COMPLETED) / COUNT(Trip WHERE status IN (COMPLETED, CANCELLED))</code>	0.20	Rewards drivers who don't generate cancelled/failed trips
On-time rate	<code>COUNT(Trip WHERE actualDuration ≤ plannedDuration × 1.15) / COUNT(Trip WHERE status=COMPLETED)</code>	0.15	Rewards predictable, on-schedule delivery
Incident-free rate	<code>1 – min(1, COUNT(Incident WHERE driverId=X AND last 90 days) / 3)</code>	0.15	Penalizes recent incidents more than old ones by using a rolling window

$\text{WeightedScore (0–1)} = 0.30 \times \text{SafetyScore} + 0.20 \times \text{LicenseCompliance} + 0.20 \times \text{CompletionReliability} + 0.15 \times \text{OnTimeRate} + 0.15 \times \text{IncidentFreeRate}$

$\text{DriverRating (stars, 1–3)} = \text{ROUND}((1 + \text{WeightedScore} \times 2) \times 2) / 2$ (maps 0→1★, 0.5→2★, 1.0→3★, rounded to nearest 0.5 for half-star precision)

Suggested display bands: 3.0 ★ green ("top performer"), 2.0–2.5 ★ amber ("standard"), below 2.0 ★ red ("needs review") — rendered as filled/half-filled/empty star icons next to the driver's name across all dashboards, not just Safety Officer's. Keep the underlying WeightedScore stored too, since a 3-point scale is coarse and not precise enough to sort or rank by on its own.

2. Vehicle Rating Score (1–3 stars)

Same idea for vehicles — a single health/performance number feeding the assignment algorithm's "maintenance risk" factor and giving the Fleet Manager and Financial Analyst a shared reference point for fleet quality.

Component	Formula (normalized 0–1)	Weight	Why
Maintenance health	<code>1 – min(1, (odometer – lastServiceOdometer) / serviceIntervalKm)</code>	0.30	Directly tracks how overdue the vehicle is for service
Utilization efficiency	<code>COUNT(Trip completed, last 30 days) / fleetAvgTripsPer30Days, capped at 1</code>	0.20	Rewards vehicles that are actually being used, not sitting idle
Fuel efficiency vs fleet	<code>vehicleDistancePerLiter / fleetAvgDistancePerLiter, capped at 1</code>	0.20	Flags vehicles burning more fuel than peers of the same type

Component	Formula (normalized 0–1)	Weight	Why
Breakdown/incident frequency	$1 - \min(1, \text{COUNT}(\text{unplanned MaintenanceLog, last 180 days}) / 3)$	0.15	Penalizes vehicles with frequent unplanned repairs
ROI contribution	$\text{normalize}(\text{VehicleFinancialSummary.roi})$ across fleet, scaled 0–1 via min-max	0.15	Ties the score back to actual profitability, not just mechanical condition

WeightedScore (0–1) = $0.30 \times \text{MaintenanceHealth} + 0.20 \times \text{UtilizationEfficiency} + 0.20 \times \text{FuelEfficiencyVsFleet} + 0.15 \times \text{BreakdownFrequency} + 0.15 \times \text{ROIContribution}$

VehicleRating (stars, 1–3) = $\text{ROUND}((1 + \text{WeightedScore} \times 2) \times 2) / 2$

Same green (3.0 ★) / amber (2.0–2.5 ★) / red (below 2.0 ★) bands as the driver score. A vehicle consistently rated below 2 ★ is a strong candidate for the Fleet Manager to flag for retirement rather than repeated repair.

3. Supporting Database Schema

Two new fields on existing entities are needed to compute the components above, plus two new tables to store computed scores (so the dashboard reads a precomputed value instead of recalculating on every page load).

3.1 New fields on existing entities

Entity	New field	Notes
Trip	plannedDuration : float (hours)	Manually estimated at trip creation, used for on-time rate
Trip	actualDuration : float (hours)	Derived: completedAt – dispatchedAt
Vehicle	lastServiceOdometer : float	Set automatically when a MaintenanceLog is closed
Vehicle	serviceIntervalKm : float	Manually configured per vehicle type (e.g. 10,000 km)
MaintenanceLog	isPlanned : boolean	Manually flagged: scheduled service vs. unplanned breakdown repair
Incident	severity : enum(LOW, MEDIUM, HIGH)	Manually entered by Safety Officer when logging an incident (new table, see prior document's incident log page)

3.2 New tables for computed scores

DriverRatingScore

id	UUID	PK
driverId	UUID	FK -> Driver.id
safetyComponent	float	-- 0-1
complianceComponent	float	-- 0-1
completionComponent	float	-- 0-1
onTimeComponent	float	-- 0-1
incidentFreeComponent	float	-- 0-1
weightedScore	float	-- 0-1, precise underlying value (for sorting/ranking)
starRating	float	-- 1.0-3.0 in 0.5 steps, for display
computedAt	datetime	

VehicleRatingScore

id	UUID	PK
vehicleId	UUID	FK -> Vehicle.id
maintenanceComponent	float	-- 0-1
utilizationComponent	float	-- 0-1
fuelEfficiencyComponent	float	-- 0-1
breakdownComponent	float	-- 0-1
roiComponent	float	-- 0-1
weightedScore	float	-- 0-1, precise underlying value (for sorting/ranking)
starRating	float	-- 1.0-3.0 in 0.5 steps, for display
computedAt	datetime	

Recompute cadence: run as a nightly batch job (or on-demand recompute when a Trip completes / MaintenanceLog closes / Incident is logged) rather than live on every request — these scores don't need to be real-time, and batching keeps the dashboard fast. Store one row per driver/vehicle and overwrite on recompute (or keep history if you want a trend line of the score over time, which is a nice bonus visualization: a small sparkline showing the last 30 days of DriverRatingScore/VehicleRatingScore next to the current number).

4. Other Places an Algorithm Could Fit

Beyond trip assignment and the two rating scores above, these are the other natural spots in TransitOps for algorithmic logic rather than pure CRUD — useful if you want to show more depth in a hackathon demo, ranked by how easy each is to build in the remaining time.

Algorithm	What it does	Build effort	Where it lives
Predictive maintenance	Estimate days/km until a vehicle's next likely failure using odometer trend + historical service interval, instead of a fixed threshold	Low	Vehicle Registry / Maintenance page, as a “predicted next service” date
Fuel/expense anomaly detection	Flag a FuelLog or Expense entry where cost-per-liter or amount is >2 standard deviations from that vehicle's/category's rolling average — catches data entry errors or possible fraud	Low	Financial Analyst dashboard, as a flagged-entries widget
Idle-vehicle reassignment suggestion	Formalizes the existing idle alert into an active suggestion: “Vehicle X has been idle 5 days while Region Y has 3 pending trips — consider relocating”	Low	Fleet Manager dashboard, as an actionable alert card
Demand forecasting	Simple moving-average or linear regression on trip volume by region/day-of-week to predict next week's demand and flag under/over-fleeting	Medium	Fleet Manager, as a forecast chart for fleet sizing decisions
Route/ETA estimation	Use distance + average speed by vehicle type (or a mapping API) to auto-estimate plannedDuration at trip creation instead of manual entry	Medium	Trip Management form, auto-filling plannedDuration used by the on-time rate KPI
Dynamic revenue suggestion	Suggest Trip.revenue based on distance × cargoWeight × a per-kg-km rate, adjustable by the Financial Analyst, instead of a blank manual field	Low	Trip Revenue & Cost page, as a pre-filled suggested value
Multi-stop load consolidation	Detect two pending trips with overlapping routes/destinations that could be merged into one dispatch to cut cost per km	High	Trip Management, as an optional “combine trips” suggestion — good stretch goal only if time allows

For an 8-hour hackathon, the highest-value additions beyond the core assignment algorithm are the fuel/expense anomaly flagging and the dynamic revenue suggestion — both are a few lines of statistics on data you're already storing, and both visibly demonstrate “intelligence” to judges without needing an external API or ML model.